The Complete Cursed Crypt Challenge

Graduate Extension: Task C: The Guarded Crypt

Implementation and Safeguards

For Task C, I added three safeguards to my Flutter signup app to handle problems with navigation and repeated submissions.

First, the Success screen checks `userName.trim().isEmpty`. If the name is blank, the app redirects to the Welcome screen instead of showing an empty greeting. I also check the name before submitting the form. This protects the Success screen even if someone opens it directly.

Next, I used an internal `_submitting` flag to block duplicate submissions. The Sign Up button is disabled while loading, but the submission method checks the flag too. A second call is ignored even if the disabled button is bypassed.

Finally, the signup method checks `mounted` after its two-second delay. If the user has already left the screen, it stops instead of trying to navigate using a disposed widget's context.

Test Results:

Test case	Expected	Observed	Result
Empty name	Redirect to Welcome; no invalid greeting	Welcome appeared; no invalid greeting or exception	Pass
Double submission	Only one Success screen	One Success screen and personalized greeting	Pass
Leave during loading	No delayed navigation or exception	Welcome remained visible; no Success screen or exception	Pass

Evidence and Acceptance Checks:

The empty-name test checks that the Success screen rejects a blank username. The double-submission test checks that repeated attempts don't create multiple Success screens. The last test checks that leaving during the loading delay doesn't cause navigation afterward. All Flutter tests passed.

Why I Chose These Safeguards

Each check protects a different point in the signup flow. The name check protects the destination screen, the `_submitting` flag protects the submission method, and `mounted` protects navigation after asynchronous work. They solve specific problems without adding extra packages or changing the existing AI Signup Coach.

Screenshot 1: Empty-name guard:



```dart
@override
void initState() {
  super.initState();

  // Task C: never display a success screen with an empty name
  if (widget.userName.trim().isEmpty) {
    WidgetsBinding.instance.addPostFrameCallback((_) {
      if (!mounted) return;

      Navigator.of(context).pushAndRemoveUntil(
        MaterialPageRoute(
          builder: (_) => const WelcomeScreen(),
        ),
        (route) => false,
      );
    });
    return;
  }

  // 🎉 Fire the confetti once when the guest arrives
  _confetti.play();
}
```

Screenshot 2: Duplicate-submission guard:



```dart
@override
void initState() {
  super.initState();

  // Task C: never display a success screen with an empty name
  if (widget.userName.trim().isEmpty) {
    WidgetsBinding.instance.addPostFrameCallback((_) {
      if (!mounted) return;

      Navigator.of(context).pushAndRemoveUntil(
        MaterialPageRoute(
          builder: (_) => const WelcomeScreen(),
        ),
        (route) => false,
      );
    });
    return;
  }

  // 🎉 Fire the confetti once when the guest arrives
  _confetti.play();
}
```

Screenshot 3: Mounted check:

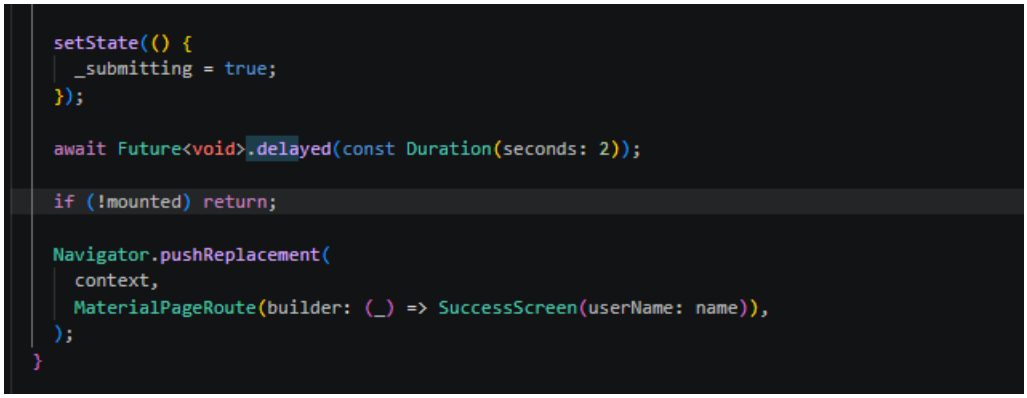
```dart
    setState(() {
      _submitting = true;
    });

    await Future<void>.delayed(const Duration(seconds: 2));

    if (!mounted) return;

    Navigator.pushReplacement(
      context,
      MaterialPageRoute(builder: (_) => SuccessScreen(userName: name)),
    );
  }
```

Screenshot 4: Hostile-case tests:



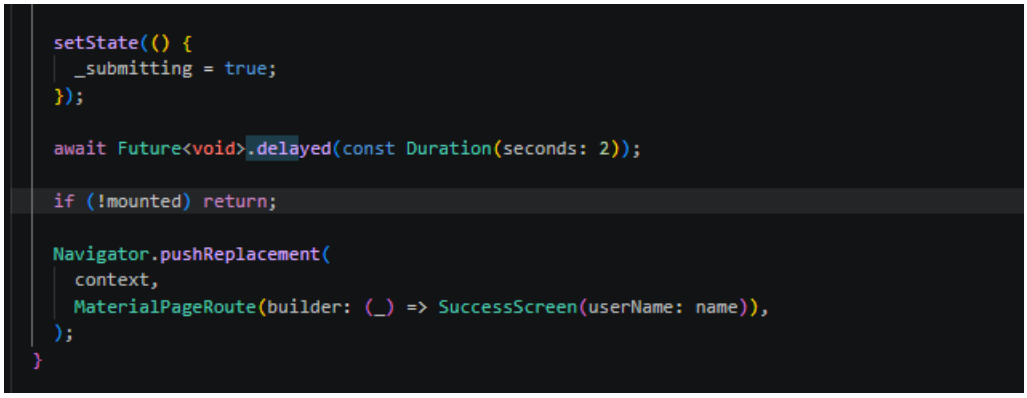
```dart
    setState(() {
      _submitting = true;
    });

    await Future<void>.delayed(const Duration(seconds: 2));

    if (!mounted) return;

    Navigator.pushReplacement(
      context,
      MaterialPageRoute(builder: (_) => SuccessScreen(userName: name)),
    );
  }
```


Screenshot 5: Test results:



```dart
    setState(() {
      _submitting = true;
    });

    await Future<void>.delayed(const Duration(seconds: 2));

    if (!mounted) return;

    Navigator.pushReplacement(
      context,
      MaterialPageRoute(builder: (_) => SuccessScreen(userName: name)),
    );
  }
```